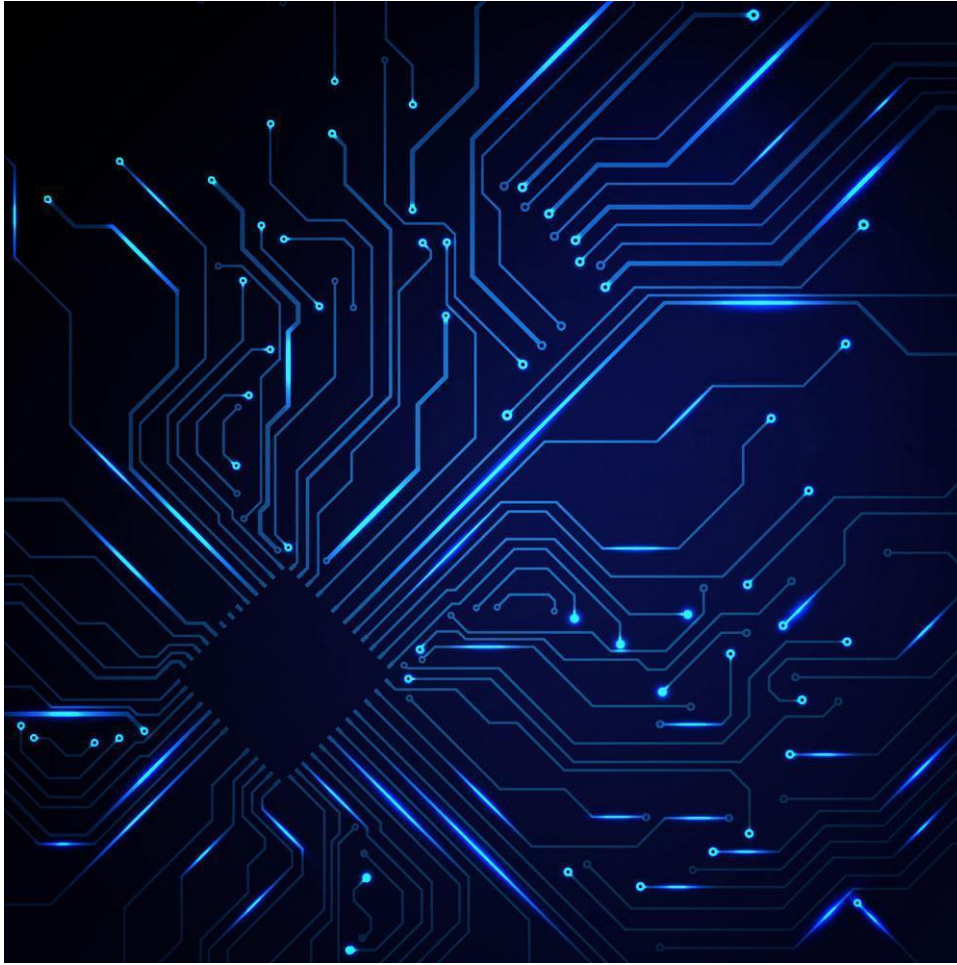




注意

1. 繳交時一律轉 PDF 檔
2. 繳交期限為
隔週三上午九點
3. 一人繳交一份
4. 檔名：學號_HW?.pdf
檔名請按照作業檔名格式進行填寫
未依照格式不予批改

2025/05/22

實驗十三

Convolution Hardware

Design

姓名：陳篁瑋

學號：01257024

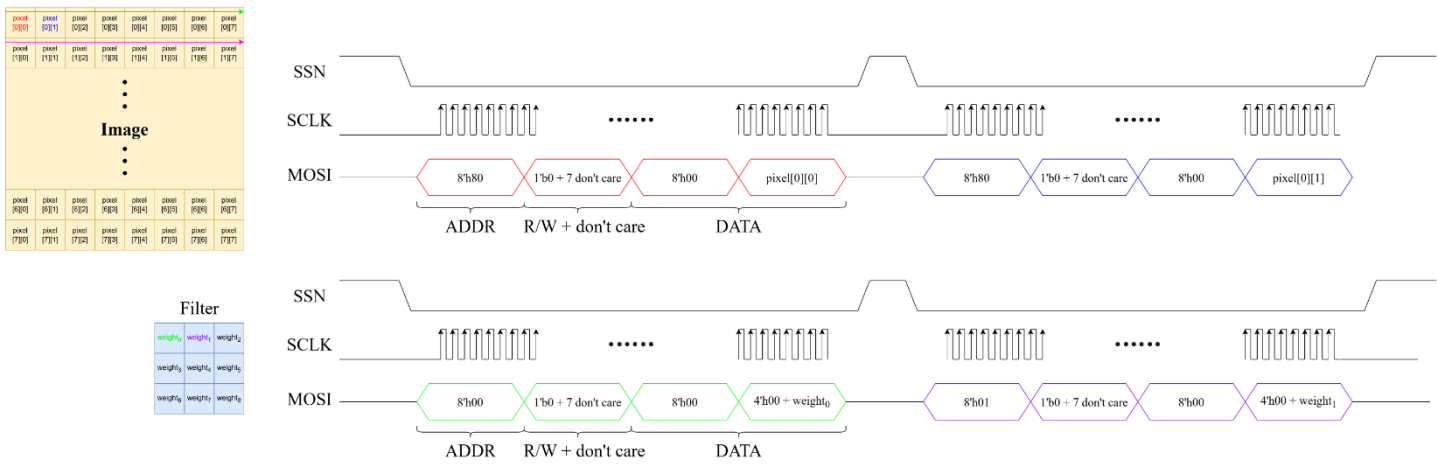
班級：資工 3A

E-mail：henry246813579@gmail.com

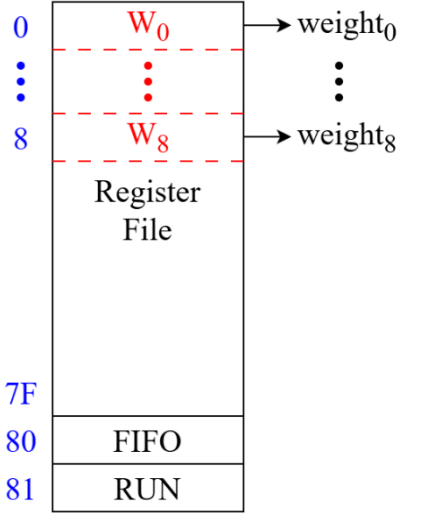
Convolution Hardware Design

■ 實驗說明：

- 1. 請根據 Convolution 演算法設計一個硬體架構。在此 LAB 中一張影像有 8×8 個像素，每個像素為 8 位元，而 filter 大小為 3×3 ，其中每個權重為 4 位元。testbench 一共 1 張影像和一個 filter。
- 2. SPI 傳輸格式



不同的 ADDR 代表不同的資料，如 ADDR 為 8h'80 表示影像資料，因其像素只有 8 個位元，故只使用到 DATA 的後 8 個位元。若 ADDR 為 8'h\$\$ (8'h00 ~ 8'h09) 則表示為 filter 資料，因其權重只有 4 個位元，故只使用到 DATA 的後 4 個位元。若 ADDR 為 8'h81 則表示影像和 filter 傳送完畢，可以觸發 CONV 開始運算。



3. CONV 輸入：

訊號	說明
sys_rst	系統重置訊號，為 1 時重置系統。
clk	系統時脈訊號，頻率為 50MHz。
sclk	串列時鐘訊號，在正緣時讀取一個位元。
ssn	當 ssn 由 1 降至 0 表傳輸開始；由 0 升至 1 則表示傳輸結束。
mosi	影像資料輸入。

4. CONV 輸出：

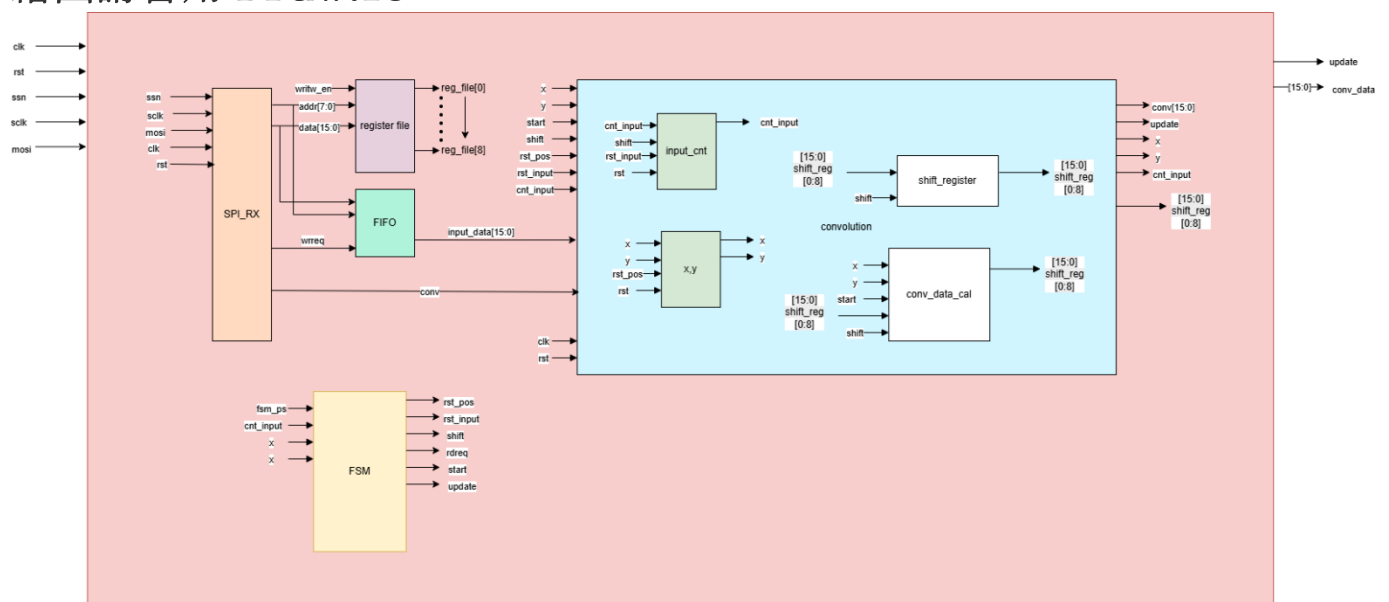
訊號	說明
update	每當 CONV 輸出一個有效像素時，發出一個時脈週期的 update 訊號。
conv_data	CONV 的像素輸出。

5. 若 ModelSim 模擬結果正確，Transcript 視窗將顯示 "Files are identical."；否則，將顯示 "Files differ."

■ 波型圖參考



■ 系統架構圖或方塊圖說明(請繪製自己設計的硬體架構圖並說明) 繪圖請善用 Draw.io



SPI_RX

負責傳入的序列資料，會判斷當前傳入的是「權重」還是「影像資料」：

- 寫入權重：透過 write_en, addr[7:0], data[15:0] 這些訊號，將卷積核的權重參數寫入上方的 register file。

- 寫入影像：透過發出 wrreq，將影像像素資料 (data) 打入下方的 FIFO。
- 控制連動：輸出一根 conv 訊號，通知 CONV 可以準備開始。

register file：專門用來存放 CNN 的 3x3 卷積核權重。

FIFO：接收來自前端的資料後，等待後端 FSM 發出讀取請求，再將 input_data[15:0] 送入運算核心。

FSM

根據條件發出全域控制訊號，包含清空座標 (rst_pos)、清空計數 (rst_input)、要求 FIFO 讀取 (rdreq)、啟動移位 (shift)、啟動運算 (start) 以及輸出有效標記 (update)。

Convolution Core

- input_cnt (輸入計數器)：接收 FSM 的 shift 訊號，計算目前已經有多少筆影像資料進入運算核心，並將 cnt_input 回傳給 FSM 作為狀態切換的依據 (例如判斷是否填滿了初始的 Line Buffer)。
- x, y (座標計數器)：追蹤目前正在處理的影像 2D 空間座標。
- shift_register (移位暫存器)：這是 CNN 硬體中最關鍵的滑動視窗 (Sliding Window) 結構。透過 shift 訊號，將不斷流進來的影像資料進行推移，維持住卷積運算所需的感受野 (Receptive Field) 資料陣列 (圖中標示為 shift_reg[0:8])。
- conv_data_cal (卷積乘加運算單元)：純數學運算單元。當收到 start 訊號，且 x, y 座標符合條件時，它會拿 shift_register 裡面的影像資料，去跟外部 register file 裡的權重進行乘加運算 (MAC)，最後計算出單一像素的卷積結果，透過 conv_data[15:0] 輸出。

- 系統架構程式碼與程式碼說明
截圖請善用 **win+shift+S**
convolution

```

1  module CNN(
2      input logic clk,
3      input logic rst,
4      input logic ssn,
5      input logic sclk,
6      input logic mosi,
7      output logic [15:0] conv_data,
8      output logic update
9  );
10
11  //SPI_RX
12  logic wrreq;
13  logic rdreq;
14  logic [15:0] data;
15  logic write_en;
16  logic [7:0] address;
17  logic conv;
18  spi_rx spi_rx_1(
19      .clk (clk),
20      .rst (~rst),
21      .ssn (ssn),
22      .sclk (sclk),
23      .mosi (mosi),
24      .write_en (write_en),
25      .wrreq (wrreq),
26      .data (data),
27      .address (address),
28      .conv (conv)
29  );
30  //register file
31  logic [15:0] reg_file [0:8];
32  always_ff@(negedge clk)
33  begin
34      if(write_en)
35          reg_file[address] <= data;
36  end
37
38  //FIFO 16 bits degree 2048
39  logic [15:0] input_data;
40  logic [10:0] usedw;
41  logic almost_empty;
42  logic sclr;
43  logic full;
44  logic empty;
45
46  fifo fifo_1(
47      .clock (clk),
48      .data (data),
49      .rdreq (rdreq),
50      .sclr (rst),
51      .wrreq (wrreq),
52      .almost_empty(almost_empty),
53      .empty (empty),
54      .full (full),
55

```

```

56         .q          (input_data),
57         .usedw      (usedw)
58     );
59
60     //CNN
61     logic rst_pos;
62     logic rst_input;
63     logic [4:0] x;
64     logic [4:0] y;
65     logic [4:0] cnt_input;
66     logic [15:0] shift_reg [0:18];
67     logic shift;
68     logic start;
69     integer i;
70     integer j;
71     always_ff@(posedge clk)
72     begin
73         if(rst) begin
74             for (i=0; i<19; i=i+1)
75                 shift_reg[i] <= 16'b0;
76         end
77         if(rst || rst_pos)begin
78             x <= 0;
79             y <= 0;
80         end
81         if(rst || rst_input)
82             cnt_input<= 0;
83         if(shift) begin
84             cnt_input <= cnt_input +1;
85             for(j=0; j<19; j=j+1) begin
86                 if(j == 18)
87                     shift_reg[18] <= input_data;
88                 else
89                     shift_reg[j] <= shift_reg[j+1];
90             end
91         end
92         if(start) begin
93             if(x<6) begin
94                 if(x==5) begin
95                     x <= 0;
96                     y <= y+1;
97                 end
98                 else
99                     x <= x+1;
100             conv_data <= shift_reg[0] *reg_file[0] +
101                 shift_reg[1] *reg_file[1] +
102                 shift_reg[2] *reg_file[2] +
103                 shift_reg[8] *reg_file[3] +
104                 shift_reg[9] *reg_file[4] +
105                 shift_reg[10] *reg_file[5] +
106                 shift_reg[16] *reg_file[6] +
107                 shift_reg[17] *reg_file[7] +
108                 shift_reg[18] *reg_file[8] ;
109
110         end

```

```

111     end
112 end
113
114 //FSM
115 typedef enum{ INIT, READ_DATA, S1, S2, S3, CNN, FINISH
116 } FSM_STATE;
117 FSM_STATE fsm_ns,fsm_ps;
118
119 always_ff@(posedge clk) begin
120     if(rst)
121         fsm_ps      <= INIT;
122     else
123         fsm_ps      <= fsm_ns;
124 end
125
126
127 always_comb
128 begin
129     fsm_ns      = fsm_ps;
130     rst_pos     = 0;
131     rst_input   = 0;
132     shift       = 0;
133     rdreq       = 0;
134     start       = 0;
135     update      = 0;
136     case(fsm_ps)
137     INIT:
138     begin
139         rst_pos     = 1;
140         rst_input   = 1;
141         if(conv)
142             fsm_ns = READ_DATA;
143     end
144     READ_DATA:
145     begin
146         shift = 1;
147         rdreq = 1;
148         if(cnt_input == 20) begin
149             fsm_ns = CNN;
150             update = 1;
151             start = 1;
152         end
153     end
154     CNN:
155     begin
156         start = 1;
157         shift = 1;
158         rdreq = 1;
159         if(y == 5 && x == 5)begin
160             update = 1;
161             fsm_ns = FINISH;
162         end
163         else if(x==5)begin
164             update = 1;
165             fsm_ns = S1;
166         end
167         else
168             update = 1;
169     end
170     S1:
171     begin
172         shift = 1;
173         rdreq = 1;
174         fsm_ns = S2;
175     end
176     S2:
177     begin
178         shift = 1;
179         rdreq = 1;
180         fsm_ns = CNN;
181     end
182     S3:
183     begin
184         shift = 1;
185         rdreq = 1;
186         fsm_ns = CNN;
187     end
188     FINISH:
189     begin
190         fsm_ns = INIT;
191     end
192 endcase
193 end
194 endmodule
195
196

```

SPI

```

1 module spi_rx(
2     input logic clk,
3     input logic mosi,
4     input logic ssn,
5     input logic sclk,
6     input logic rst,
7     output logic [7:0] address,
8     output logic [15:0] data,
9     output logic wrreq,
10    output logic write_en,
11    output logic conv
12 );
13
14
15 //Edge Detector
16 logic sclk_s;
17 logic sclk_d;
18 logic ssn_s;
19 logic ssn_d;
20 logic ssn_negedge;
21 logic sclk_posedge;
22 logic sclk_negedge;
23
24 always_ff@(posedge clk)
25 begin
26     if(~rst)
27     begin
28         sclk_s      <= 1'b0;
29         sclk_d      <= 1'b0;
30         ssn_s       <= 1'b1;
31         ssn_d       <= 1'b1;
32     end
33     else
34     begin
35         {sclk_d, sclk_s} <= {sclk_s, sclk};
36         {ssn_d, ssn_s}  <= {ssn_s, ssn};
37     end
38 end
39
40 assign ssn_negedge = ~ssn_s & ssn_d;
41 assign sclk_posedge = sclk_s & ~sclk_d;
42 assign sclk_negedge = ~sclk_s & sclk_d;
43
44 //Counter
45 logic [31:0] receive_data_counter;
46 logic sclk_data_counter_rst;
47
48 always_ff@(posedge clk)
49 begin
50     if(~rst || ssn)
51         receive_data_counter <= 32'b0;
52     else if(sclk_posedge)
53         receive_data_counter <= receive_data_counter + 1'b1;
54 end
55

```

```

56 //Counter2
57 logic [5:0] sclk_data_counter;
58
59 always_ff@(posedge clk)
60 begin
61     if(~rst || sclk_data_counter_rst)
62         sclk_data_counter <= 32'b0;
63     else if(sclk_negedge)
64         sclk_data_counter <= sclk_data_counter + 1'b1;
65 end
66
67 //Shift Register
68 logic [31:0] shift_data;
69 logic shift_read_data;
70 logic load_shift_read_data;
71 logic [15:0] read_data_reg;
72 logic [15:0] read_data;
73
74 always_ff@(posedge clk)
75 begin
76     if(~rst)
77         shift_data <= 32'b0;
78     else if(ssn_negedge)
79         shift_data <= 32'b0;
80     else if(~ssn && sclk_posedge)
81         shift_data <= {shift_data[30:0], mosi};
82     if(~rst)
83         read_data_reg <= 16'b0;
84     else if(load_shift_read_data)
85         read_data_reg <= read_data;
86     else if(shift_read_data && sclk_negedge) begin
87         read_data_reg <= read_data_reg << 1;
88     end
89 end
90
91 //input counter
92 logic input_flag;
93 logic [8:0] input_cnt;
94
95 always_ff@(posedge clk)
96 begin
97     if(~rst)
98         input_cnt <= 0;
99     else if(input_flag)
100         input_cnt <= input_cnt +1;
101 end
102
103 //reg
104 logic load_command;
105 logic load_address;
106 logic load_data;
107 logic command;
108
109 always_ff@(posedge clk)
110

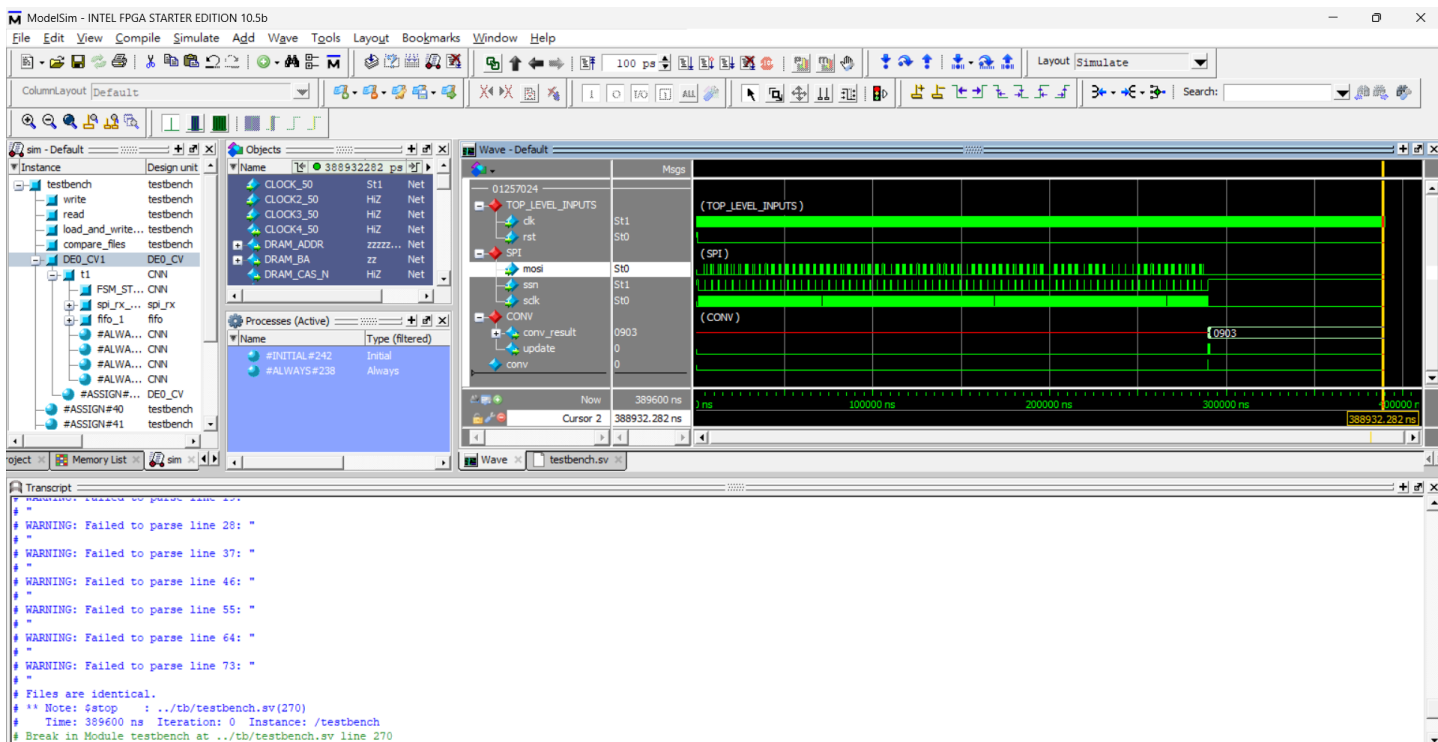
```

```

111 begin
112   if(load_command)
113     command <= shift_data[7];
114   if(load_address)
115     address <= shift_data[15:8];
116   if(load_data)
117     data <= shift_data[15:0];
118   end
119
120   //FSM
121   typedef enum{ INIT, START_SPI_RX, RECEIVE_ADDRESS, DUMMY,
122     CHECK_COMMAND, TX_REQ, FINISH, RECEIVE_DATA, WRITE
123   } FSM_STATE;
124   FSM_STATE fsm_ns,fsm_ps;
125
126   always_ff@(posedge clk) begin
127     if(~rst)
128       fsm_ps      <= INIT;
129     else
130       fsm_ps      <= fsm_ns;
131   end
132
133   always_comb
134   begin
135     fsm_ns          = fsm_ps;
136     load_command    = 0;
137     load_address    = 0;
138     load_data       = 0;
139     write_en        = 0;
140     wrreq           = 0;
141     input_flag      = 0;
142     conv            = 0;
143     case(fsm_ps)
144       INIT:
145       begin
146         fsm_ns = START_SPI_RX;
147       end
148       START_SPI_RX:
149       begin
150         if(ssn_negedge) begin
151           fsm_ns = RECEIVE_ADDRESS;
152         end
153         else
154           fsm_ns = START_SPI_RX;
155       end
156       RECEIVE_ADDRESS:
157       begin
158         if(receive_data_counter==16) begin
159           load_command = 1;
160           load_address = 1;
161           fsm_ns = CHECK_COMMAND;
162         end
163       end
164       CHECK_COMMAND:
165       begin
166         fsm_ns = RECEIVE_DATA;
167       end
168       RECEIVE_DATA:
169       begin
170         if(receive_data_counter==32)begin
171           load_data = 1;
172           fsm_ns = WRITE;
173         end
174         else
175           fsm_ns = RECEIVE_DATA;
176       end
177       WRITE:
178       begin
179         if(address>=10)
180           wrreq = 1;
181         else
182           write_en = 1;
183           input_flag = 1;
184           if(input_cnt == 73)
185             conv = 1;
186           fsm_ns = FINISH;
187       end
188       FINISH:
189       begin
190         fsm_ns = INIT;
191       end
192     endcase
193   end
194 endmodule
195
196
197
198

```

■ 模擬結果與結果說明(需截到 Transcript 視窗)：



■ 結論與心得：

這次比前兩個友善多了，感覺做過 **TDI** 後做 **CNN**，感覺起來很類似的作法，只是核心的 **kernel** 有改，和 **SPI** 要改而已。